Parameter-Efficient Finetuning (PEFT)

- Prefix tuning
- Adapter
- Low rank adaptation (LoRA)

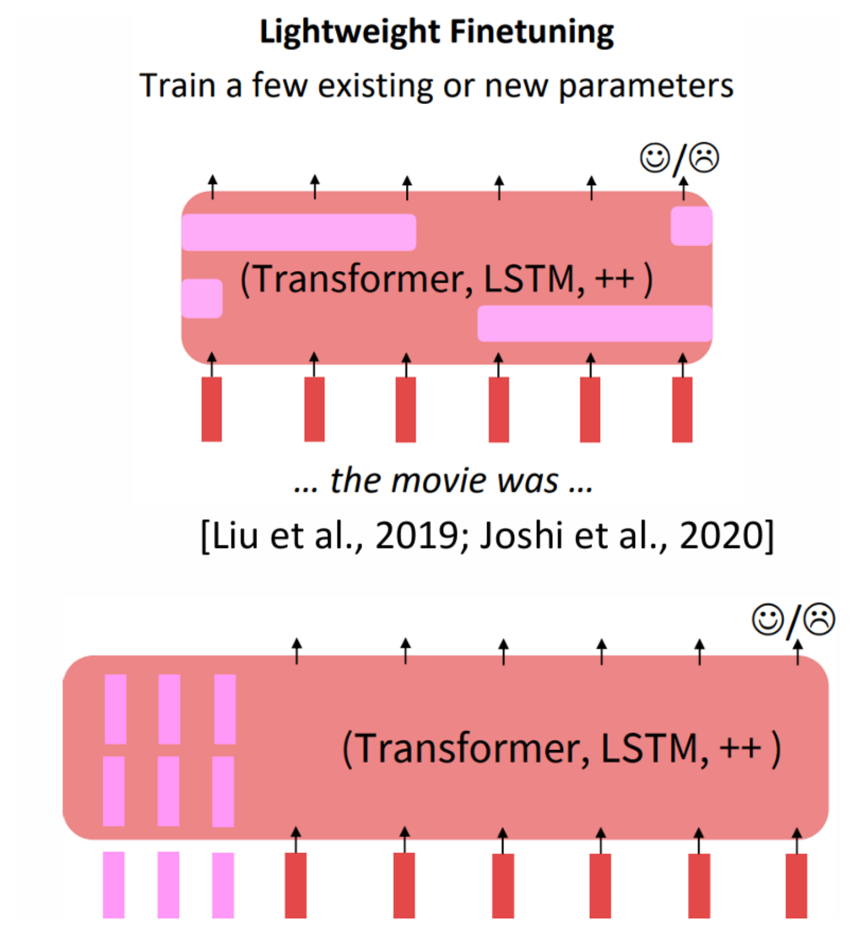
Full finetuning vs Lightweight finetuning

Lightweight fine tuning: Fine tuning on a subset of the parameters

Prefix tuning

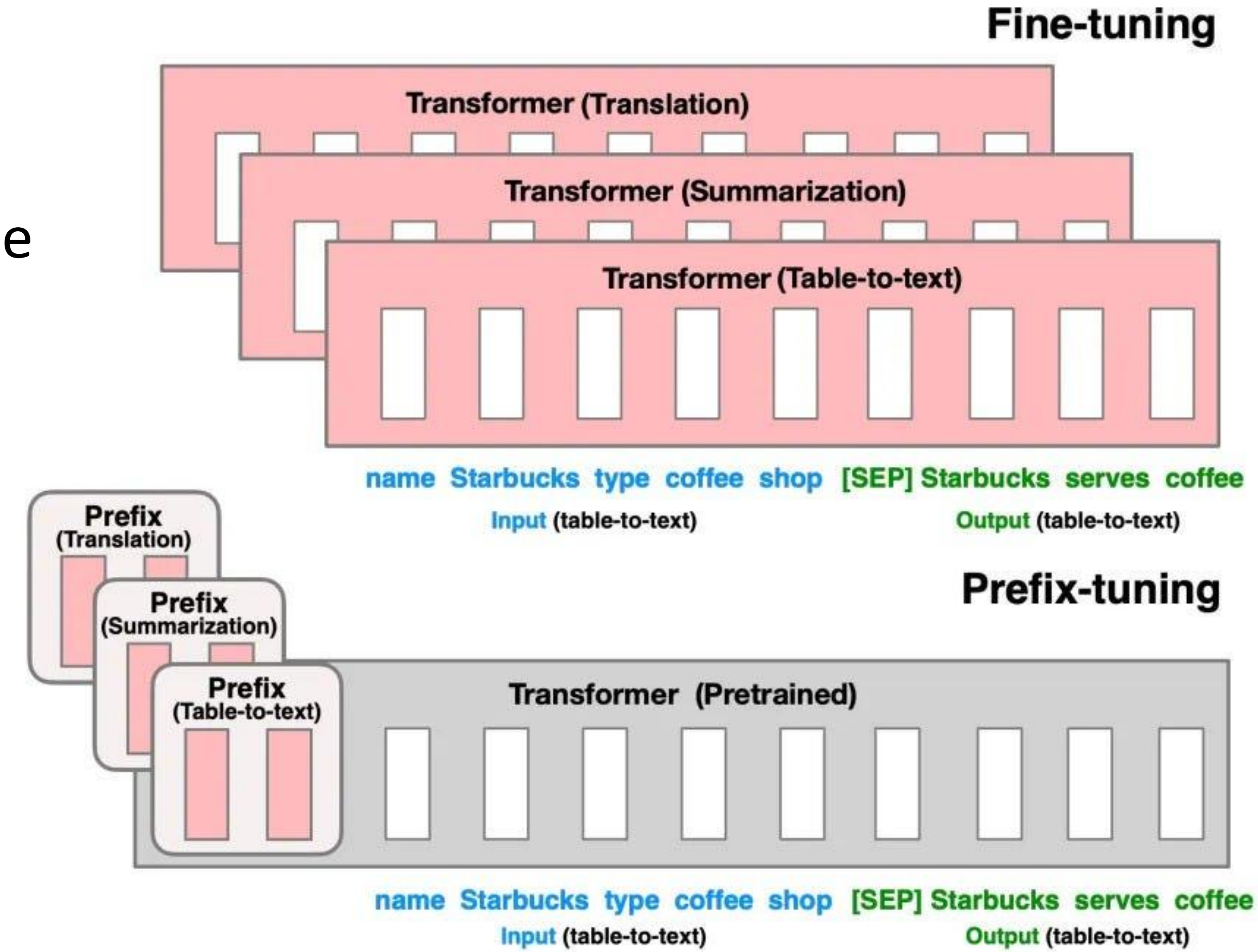
Adapter

LoRA

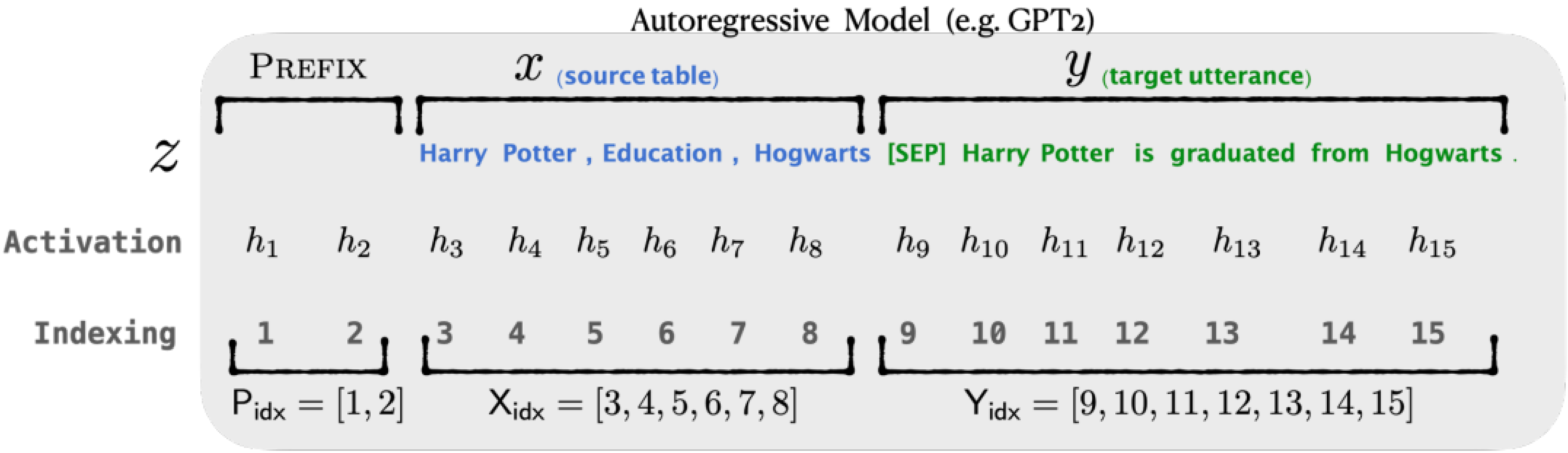


Prefix tuning

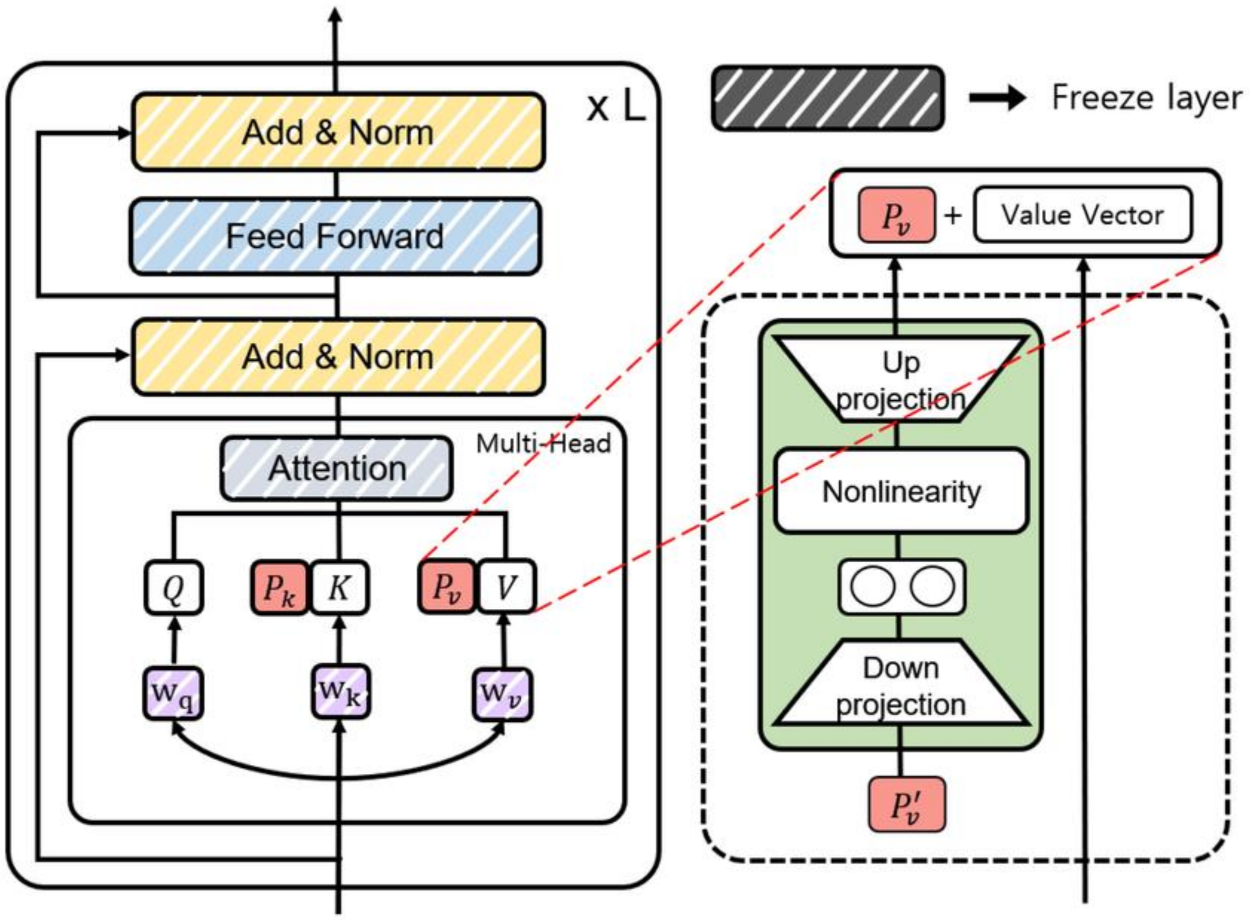
- Freez all params
- Prepend prefix tokens to the input and layers



Prefix tuning



Prefix tuning



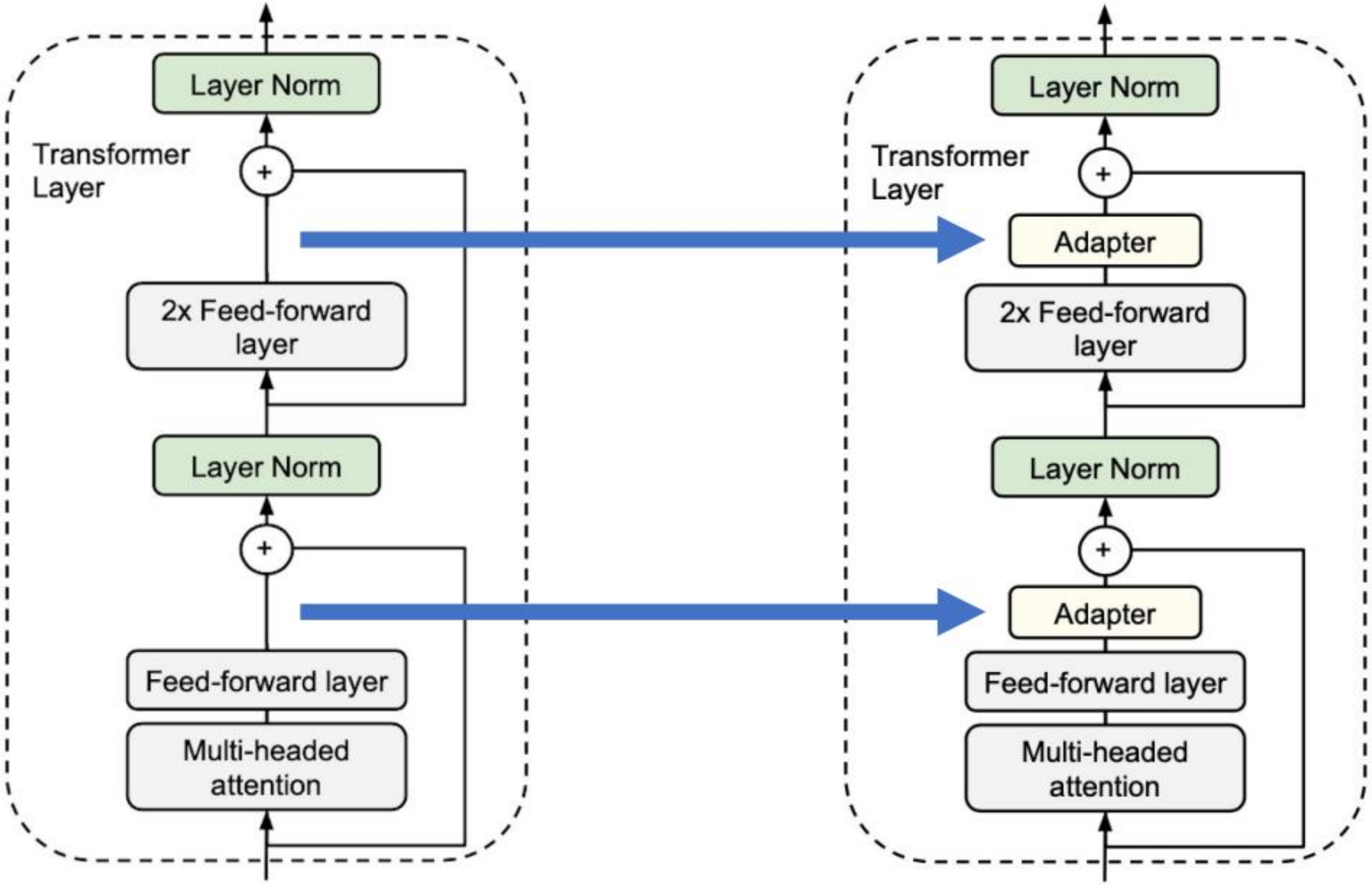
Adapter

- Introduced in ICML 2019.

Parameter-Efficient Transfer Learning for NLP

Neil Houlsby [1] Andrei Giurgiu [1 *] Stanisław Jastrzębski [2 *] Bruna Morrone [1] Quentin de Laroussilhe [1]
Andrea Gesmundo [1] Mona Attariyan [1] Sylvain Gelly [1]

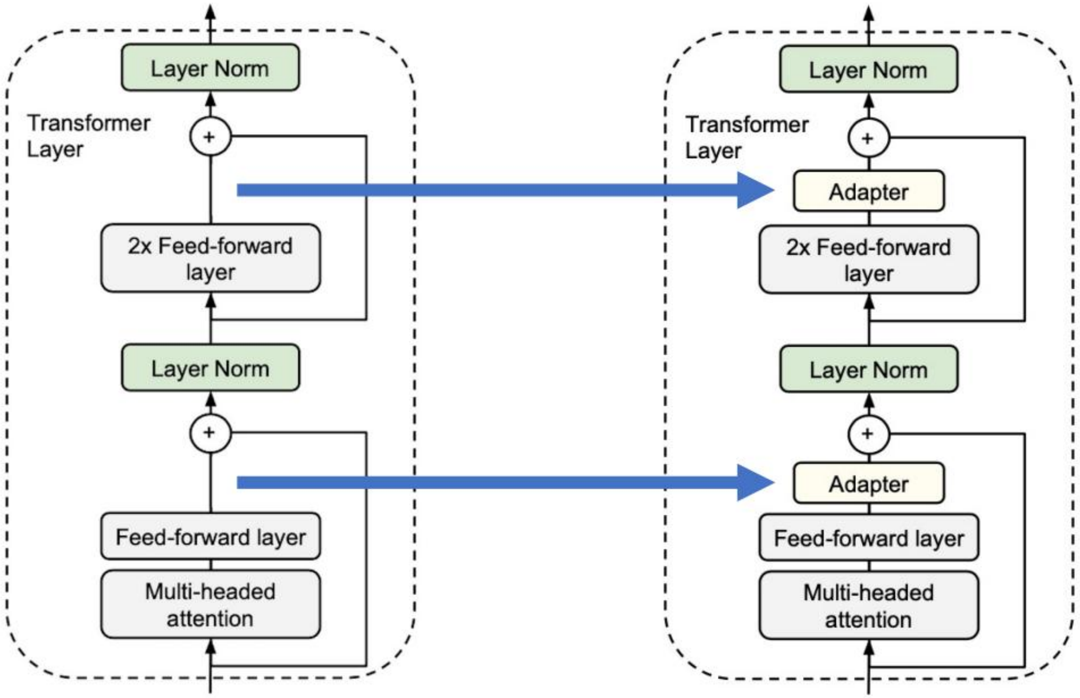
Adapter

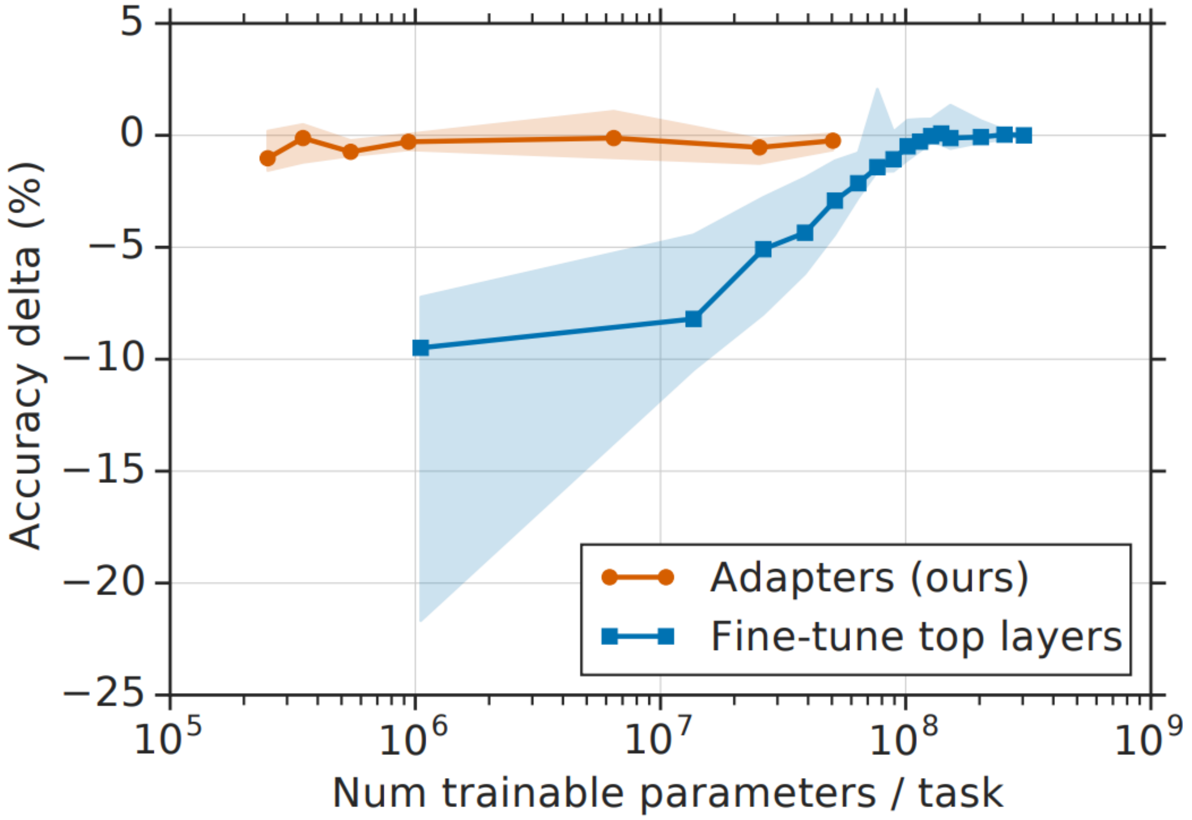


Layer Norm
Transformer Layer
+
2x Feed-forward layer
Layer Norm
+
Feed-forward layer
Multi-headed attention
Layer Norm
Transformer Layer
+
Adapter
2x Feed-forward layer
Layer Norm
+
Adapter
Feed-forward layer
Multi-headed attention

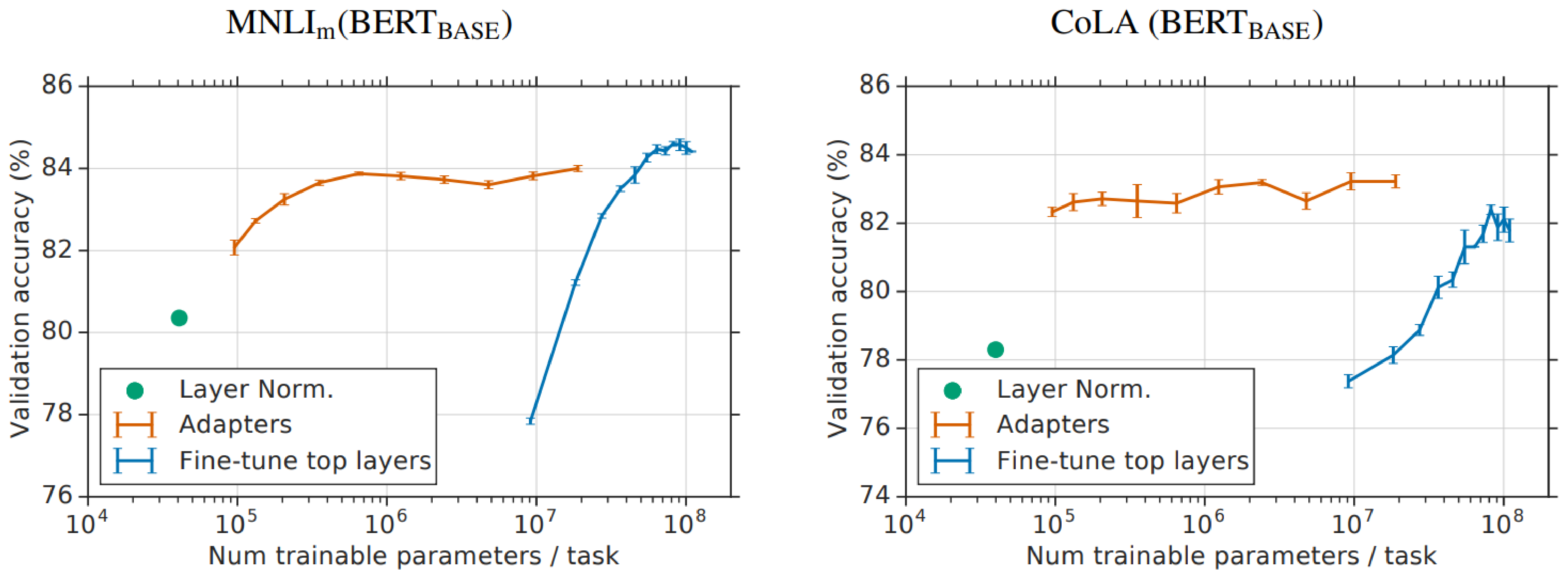
Adapter

- %0.5 - %8   of the total parameters.
- No need to store a full model for each task/user





FT vs Adapter



Low Rank Adaptation - Motivation

Want to read more?

Aghajanyan, et al. **Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning(2020)**

Intrinsic Dimension Hypothesis:

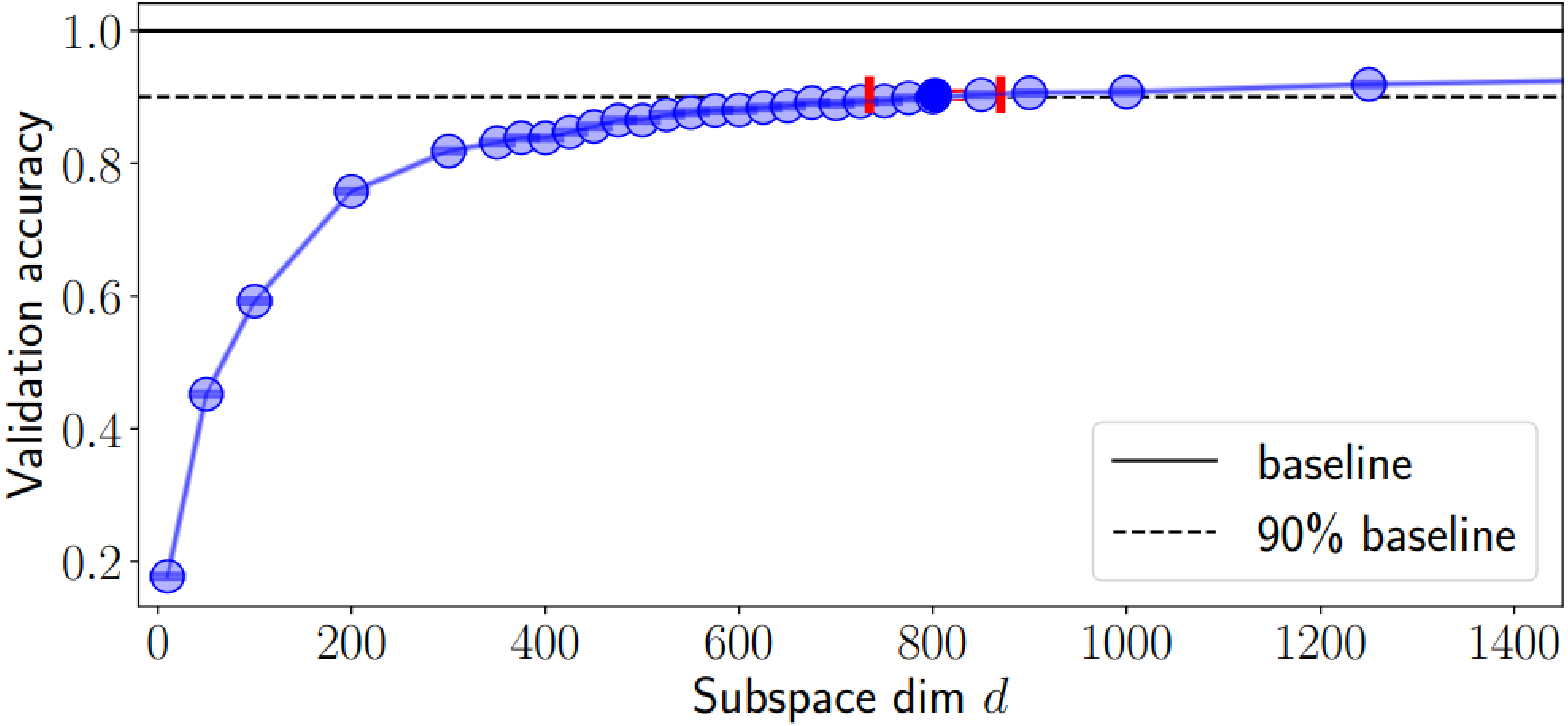
Neural network adaptation exhibits low intrinsic rank,

meaning updates Δ W do not require full-rank space to alter behavior effectively

LoRA: weight updates during task adaptation reside in a **much lower** intrinsic dimension than the original parameter space

LoRA – Motivation

Want to read more?
Aghajanyan, et al. **Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning(2020)**



LoRA – Motivation

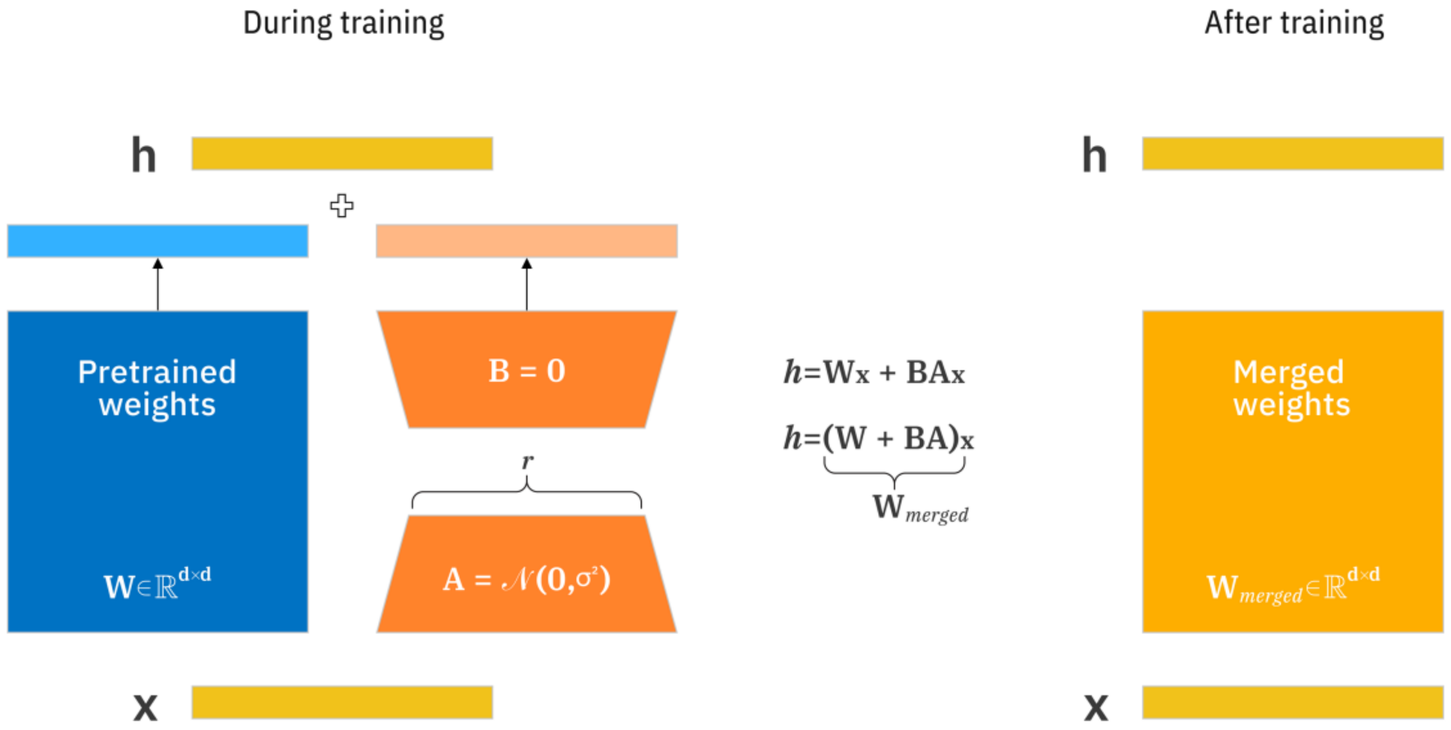
Want to read more?

Aghajanyan, et al. **Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning(2020)**

	SAID		DID	
Model	MRPC	QQP	MRPC	QQP
BERT-Base	1608	8030	1861	9295
BERT-Large	1037	1200	2493	1389
RoBERTa-Base	896	896	1000	1389
RoBERTa-Large	**207**	**774**	322	**774**

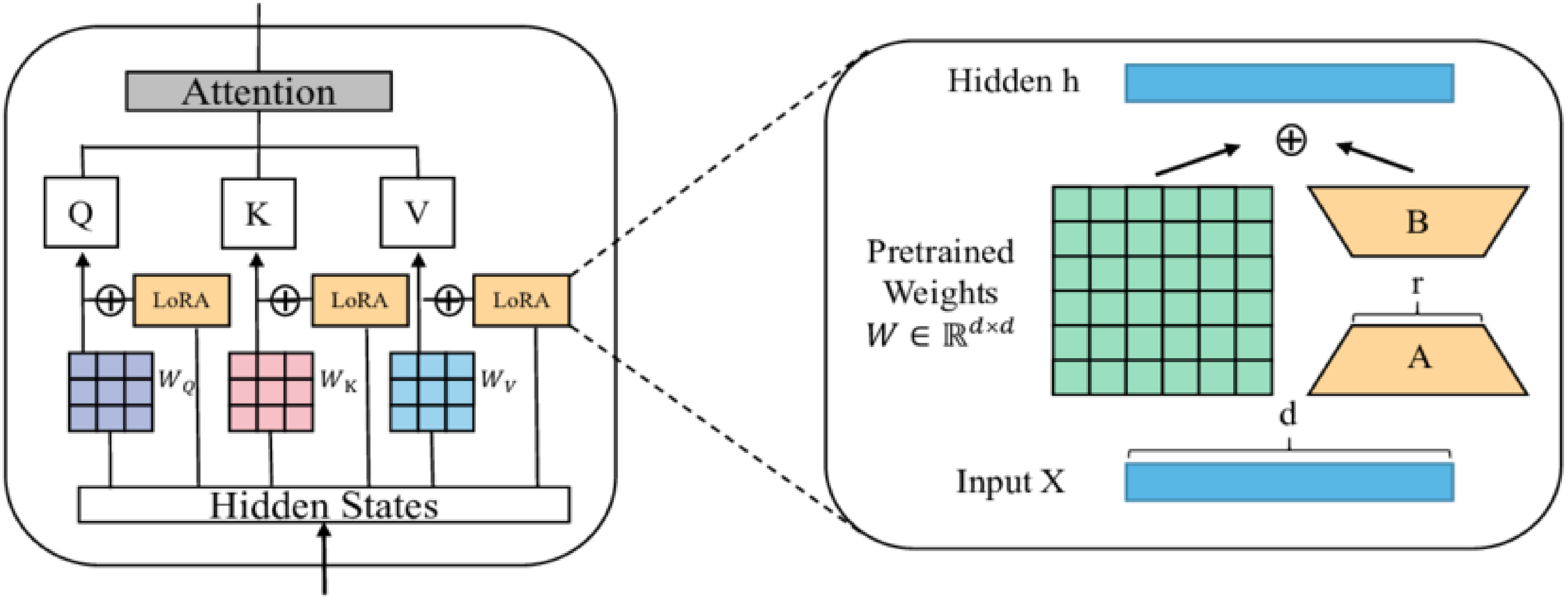
Table 1: Estimated d_{90} intrinsic dimension for a set of sentence prediction tasks and common pre-trained models. We present both the *SAID* and *DID* methods.

LoRA


During training
After training
h
Pretrained weights
$W \in \mathbb{R}^{d \times d}$
$B = 0$
r
$A = \mathcal{N}(0, \sigma^2)$
x
$h = Wx + BAx$
$h = \underbrace{(W + BA)}_{W_{merged}}x$
h
Merged weights
$W_{merged} \in \mathbb{R}^{d \times d}$
x

LoRA in transformer

- LoRA could replace in every layer but original paper only applies in attention weights
- GPT3 : only key and query (Empirically)



Multi-Head Attention

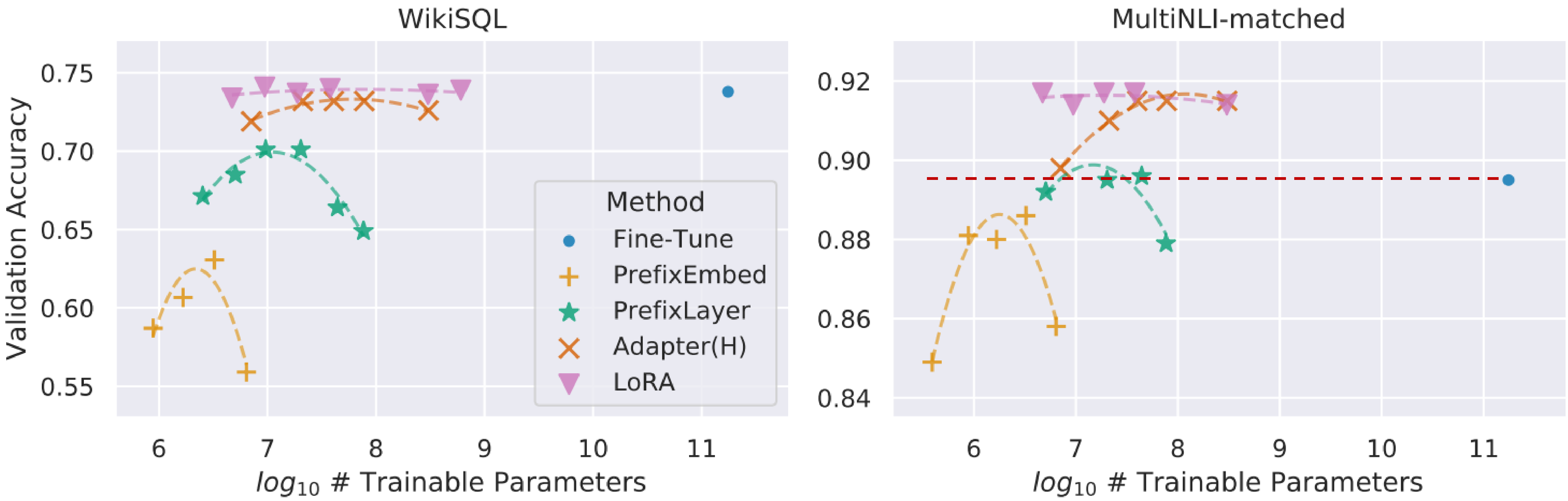
LoRA

LoRA – Result

	# of Trainable Parameters = 18M						
Weight Type Rank r	W_q 8	W_k 8	W_v 8	W_o 8	W_q, W_k 4	W_q, W_v 4	W_q, W_k, W_v, W_o 2
WikiSQL ($\pm 0.5\%$)	70.4	70.0	73.0	73.2	71.4	**73.7**	**73.7**
MultiNLI ($\pm 0.1\%$)	91.0	90.8	91.0	91.3	91.3	91.3	**91.7**

Table 5: Validation accuracy on WikiSQL and MultiNLI after applying LoRA to different types of attention weights in GPT-3, given the same number of trainable parameters. Adapting both W_q and W_v gives the best performance overall. We find the standard deviation across random seeds to be consistent for a given dataset, which we report in the first column.

LoRA – Result



WikiSQL
MultiNLI-matched
Validation Accuracy
0.75
0.70
0.65
0.60
0.55
0.92
0.90
0.88
0.86
0.84
6
7
8
9
10
11
log_{10} # Trainable Parameters
Method
Fine-Tune
PrefixEmbed
PrefixLayer
Adapter(H)
LoRA

LoRA – Result

Model&Method	# Trainable Parameters	WikiSQL	MNLI-m	SAMSum
		Acc. (%)	Acc. (%)	R1/R2/RL
GPT-3 (FT)	175,255.8M	**73.8**	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 (Adapter$^{\mathrm{H}}$)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 (Adapter$^{\mathrm{H}}$)	40.1M	73.2	**91.5**	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	**91.7**	**53.8/29.8/45.9**
GPT-3 (LoRA)	37.7M	**74.0**	**91.6**	53.4/29.2/45.1

Table 4: Performance of different adaptation methods on GPT-3 175B. We report the logical form validation accuracy on WikiSQL, validation accuracy on MultiNLI-matched, and Rouge-1/2/L on SAMSum. LoRA performs better than prior approaches, including full fine-tuning. The results on WikiSQL have a fluctuation around $\pm 0.5\%$, MNLI-m around $\pm 0.1\%$, and SAMSum around $\pm 0.2/\pm 0.2/\pm 0.1$ for the three metrics.

Your turn

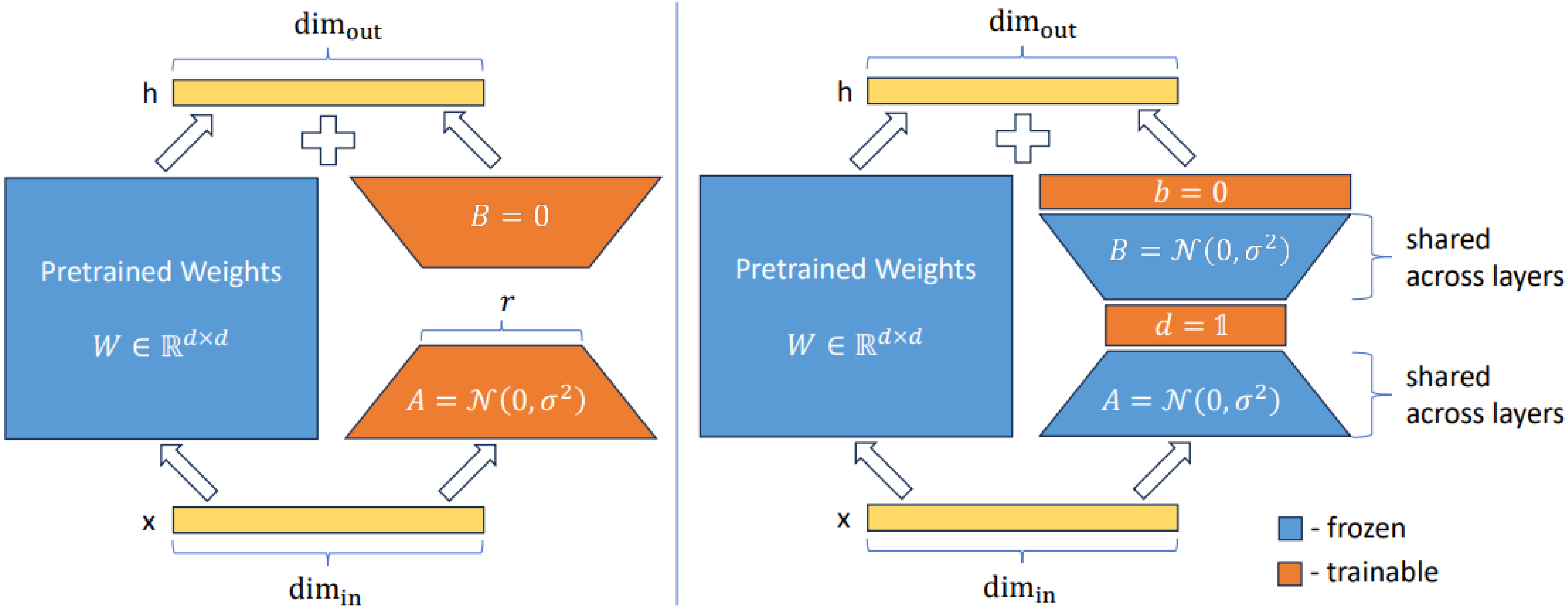
What is the biggest advantage of LoRA?

- Lower Memory Usage?
- Cheaper Hardware?
- Preserved Base Knowledge?

LoRA Variants

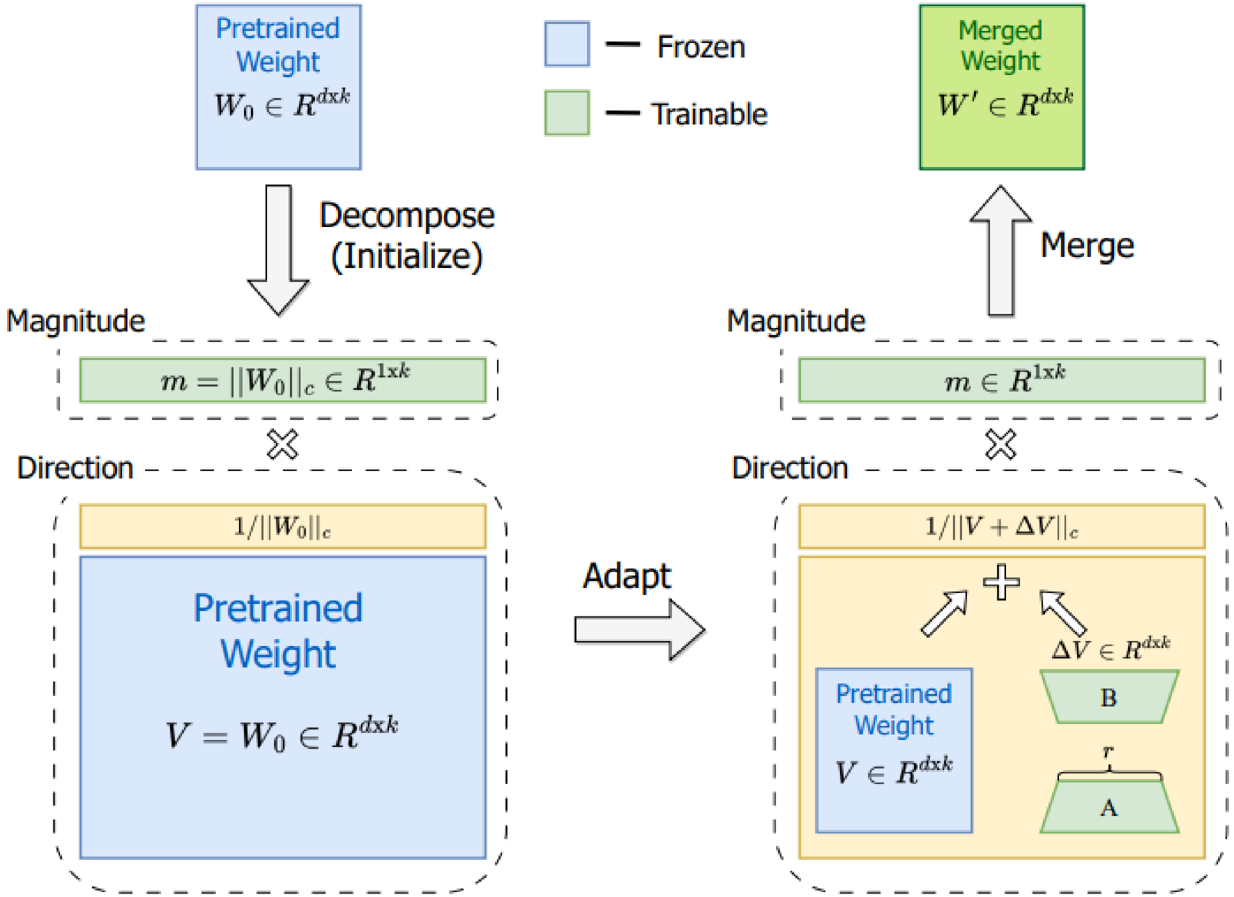
DailyDoseofDS.com

LoRA Variants - VeRA



VeRA: Vector-based Random Matrix Adaptation, ICLR 2024

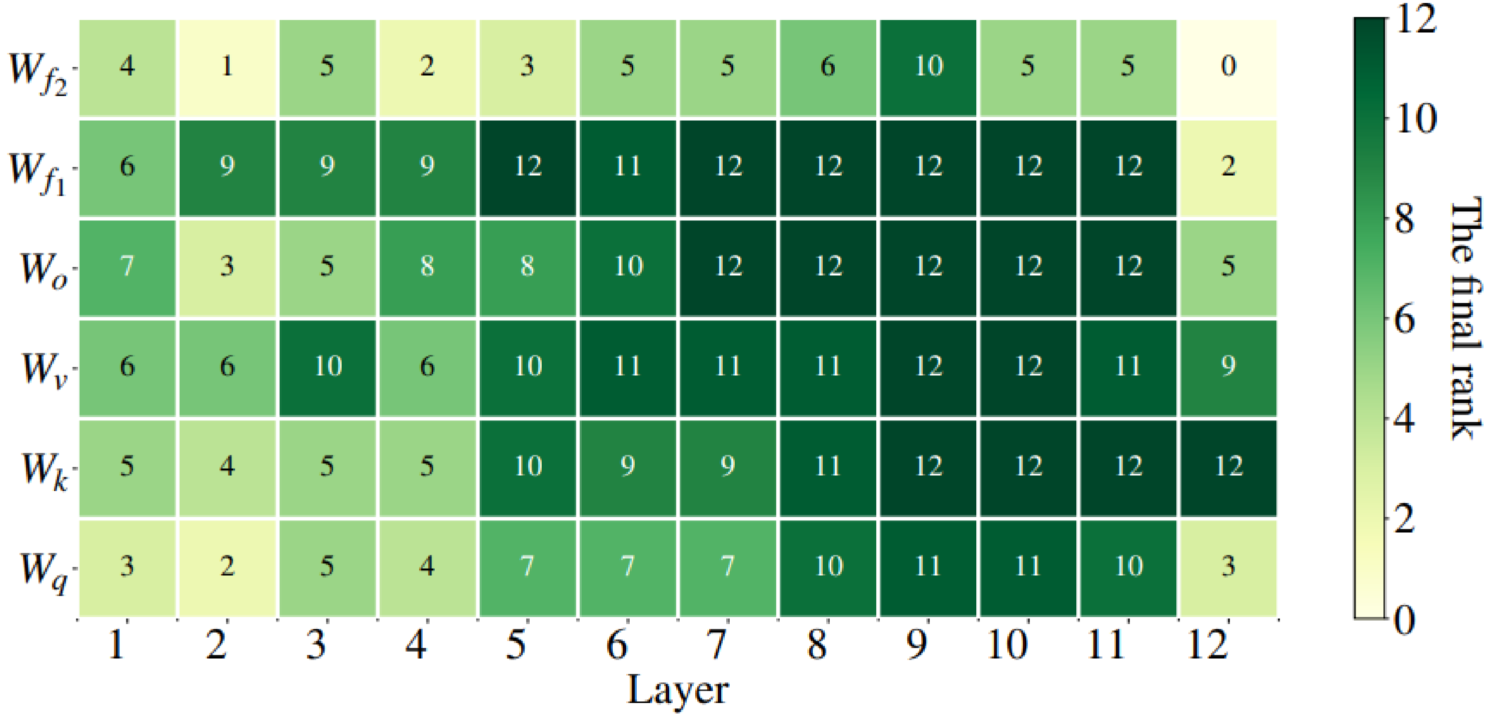
LoRA Variants - DoRA



DoRA: Weight-Decomposed Low-Rank Adaptation, ICML 2024

LoRA Variants - AdaLoRA

- **Ada**ptive **LoRa**: adapt the rank of the LoRA matrices dynamically.

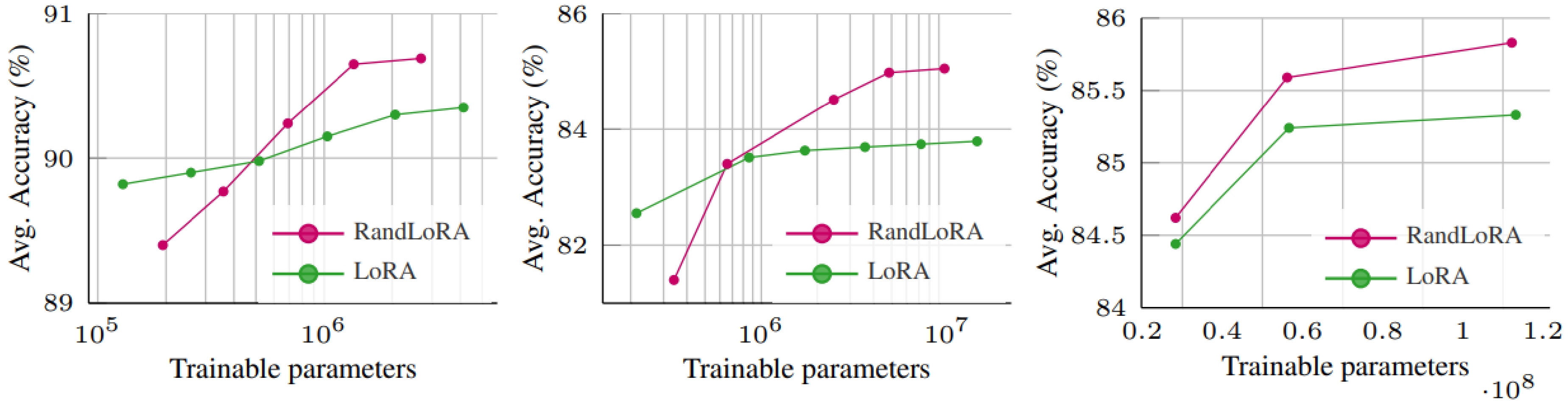


AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning, ICLR, 2023

LoRA Variants - RandLoRA

- full-rank updates using a learned linear combinations of low-rank, non-trainable random matrices

$$\Delta W = \sum_{j=1}^{n} B_j \Lambda_j A_j \Gamma_j ,$$



(a) DinoV2 (b) CLIP (c) LLama3-8B

RandLoRA: Full-rank parameter-efficient fine-tuning of large models, ICLR 2025